

Marketplace Paid Promotion Feature Analysis

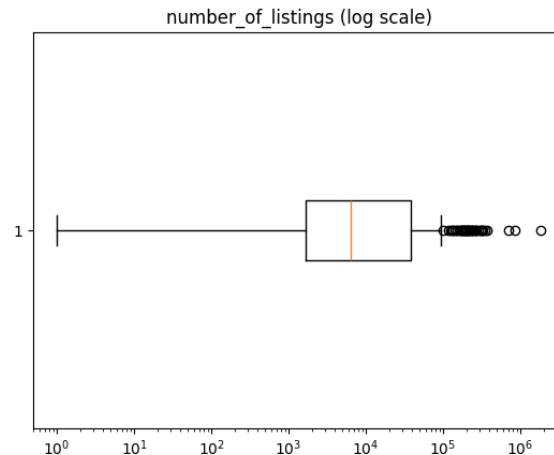
1 Section I: Evaluate data quality

1.1 Exploratory analysis of the dataset, observations. Dataset description and interesting features

```
df.head()
```

	category_2	category_3	number_of_listings	avg_listing_price_eur	revenue_from_push_ups
0	ACCESSORIES_JEWELLERY	ACCESS_MOBILE	14228	28.081851	492.0
1	ACCESSORIES_JEWELLERY	BELTS	29838	21.273161	1382.0
2	ACCESSORIES_JEWELLERY	GLOVES	6350	17.043860	304.0
3	ACCESSORIES_JEWELLERY	JEWELRY	343493	17.935880	20554.0
4	ACCESSORIES_JEWELLERY	KEY_RINGS	5	15.900000	NaN

Given dataset is 5 columns and 238 rows. I immediately check the very top row to see the titles of the columns to understand what is the data about. What seems a bit unclear to me are titles `category_2` and `category_3`. I assume there is `category_1` because there are `category_2` and `category_3`. These titles would need to change to provide more clarity and make it easier to work with the data. For example, for now I'd rename `category_1` to `category`, `category_2` to `sub_category`, and `category_3` to `sub_sub_category`, but even this might not have optimal clarity especially if there are multiple subcategories. Now at least we have a more clear idea that there is a hierarchy of categories and names that are easier to work with, but there is still room for improvement. Other titles seem good and clear enough to work with. Now, moving on to the data itself. I notice some inconsistent naming: there are names that are shortened but not as clear (`WOM_COS_HAN_COSMETICS`) and there are names that are long but clear (`TECHNICAL_ITEMS_FOR_BABIES_AND_KIDS`). These names should be consistent so, personally, I'd choose long and clear variant, because I prefer clarity. In the `number_of_listings` I notice that most of the values are in the hundreds, thousands or tens of thousands so these are fine. The outliers are in the hundred of thousands or one value even above a million, so this could mean that there should be more categories and subcategories to make it more granular and even it out. Categories that have below 10 listings may be too granular and those categories could be connected instead. There is a big difference between max. value and min. value so this should make it better.



Looking at the `avg_listing_price_eur` column I see no need to have so many numbers after the decimal point when talking about money so I would change it to 3 or 4 numbers after the decimal point. `avg_listing_price_eur` might not be enough. It would be good to also have `median_listing_price_eur` because median value is less affected by outliers and skewed data (very low price items or unreasonably high price items), providing a more accurate representation. At the end, obviously, do data imputation with missing or null values with whatever method fits best.

1.2 How would you perform testing on this dataset to ensure its quality?

Create test cases and test data that cover all possible scenarios where data may be missing, incomplete or inaccurate. Define rules for each column and create test cases from that, for example, `number_of_listings`, `avg_listing_price_eur`, `revenue_from_push_ups` can't have a negative value, test for edge cases. Check for NULL values and replace them. Check for outliers before calculating `avg_listing_price_eur` to avoid skewing the data. You can also have some sort of a buffer, acceptable error rates, tolerances so not all outliers get discarded. Currently, `revenue_from_push_ups` displays NULL if no one from that category used a push-up. Instead of that, simply make 0 the default value to avoid any unnecessary issues.

```
clean_df = df.fillna({"revenue_from_push_ups": 0})
clean_df.sort_values('revenue_from_push_ups')
```

	category_2	category_3	number_of_listings	avg_listing_price_eur	revenue_from_push_ups
4	ACCESSORIES_JEWELLERY	KEY_RINGS	5	15.900000	0.0
12	ACCESSORIES_JEWELLERY	W_ACC_UMBRELLAS	1	5.000000	0.0
58	CHILD_SAFETY	BAB_CHI_OTHER	48	18.926042	0.0
47	BAGS_BACKPACKS	W BUM_BAGS	1	7.900000	0.0
48	BAGS_BACKPACKS	W_HOLDALLS	7	26.428571	0.0
...	...	...	...	...	...
105	FOOTWEAR	W_TRAINERS	289412	50.490460	43358.0
228	WOMENS	PULLOVERS_SWEATERS	713152	14.512787	47128.0
227	WOMENS	OUTERWEAR	377058	46.638132	57032.0
235	WOMENS	TOPS_T_SHIRTS	1844512	10.469959	76166.0
222	WOMENS	DRESSES	860056	39.783769	79424.0

238 rows x 5 columns

All of these and other predefined rules help the data stay consistent. If there is a big number of listings for a category, you could split it into, for example, 3 blocks, calculate the average listing price for each block and compare each other. If there are records from a year ago or some time ago and you know they're accurate, you can compare current values with those previous values and check for abnormalities, tendencies or accuracy. These methods, other statistical methods or specific tools help you test your data's accuracy. Check for any duplicates.

1.3 Granularity of the dataset

This dataset does not display a lot of data so the granularity of this dataset is quite low. It doesn't show every listing and every listing's price or even more, but it shows listings from broad categories and the average price, revenue from push-ups of all of the listings in that one category. Personally, I'd make it a little bit more granular so it's easier to work with. I would add more categories and subcategories, because right now some of them have hundreds of thousands of listings while others thousands or tens of thousands. I'd connect a few categories because they have lower than 10 listings. I would also add `median_listing_price_eur` because median value is less affected by outliers and skewed data, providing a more accurate representation. Also, the granularity of the dataset is time-based. In a big marketplace, number of listings, average listing price, and revenue from push-up can change all the time: weekly, daily, hourly or even by the minute.

1.4 Descriptions for the columns in the dataset

- `"category_2"` - broad and general names of item categories that are used as a hierarchical top for searching (string)
- `"category_3"` - subcategory names that specify their respective parent's category items (string)
- `"number_of_listings"` - a number of total amount of listings in one specific subcategory (integer+)
- `"avg_listing_price_eur"` - average price in euros of one listing in one specific subcategory (float+)
- `"revenue_from_push_ups"` - a number of total amount of revenue in euros from push-ups in one specific subcategory (float+)

Comments: I included data types in each column description so there is no confusion about types. Regular strings, integers, floats, and '+' sign that represents only non-negative integers or non-negative floats. Included currency types in those columns that represent currencies for clarity.

2 Section II: Evaluate current situation

2.1 Based on the provided data, what metric can you define to check if users are interested in this feature and why?

To check if the users are interested in the push-up feature, I can define a `push_up_rate` metric. In our data we have `revenue_from_push_ups` for each subcategory and we know that the push-up feature costs 2eur. For each subcategory we can get `number_of_push_ups = revenue_from_push_ups / 2`. After calculating this for each subcategory, we take `number_of_listings` from the dataset for each subcategory. Now, we can get `push_up_rate = (number_of_push_ups/number_of_listings)*100` and we get a wide variety of rates from 0% up to 18%. We have to notice that sample-size of the top push-up rate is only 33 listings, which is not a lot, so it should be interpreted cautiously.

```
clean_df2.sort_values('push_up_rate', ascending=False)
```

	number_of_listings	avg_listing_price_eur	revenue_from_push_ups	push_up_rate
168	33	82.545455	12.0	18.181818
207	2544	198.490248	470.0	9.237421
209	19311	224.519087	3486.0	9.025944
34	4118	75.345675	726.0	8.814959
162	2305	96.893262	398.0	8.633406
...	...	...	...	...
4	5	15.900000	0.0	0.000000
12	1	5.000000	0.0	0.000000
92	13	13.273077	0.0	0.000000
172	61	23.155738	0.0	0.000000
125	2594	9.851823	0.0	0.000000

238 rows x 4 columns

You can also calculate the total `push_up_rate` (all categories combined): `number_of_push_ups.sum() / number_of_listings.sum() * 100` $\approx 3.20555\%$. Since the feature costs money, the purchase rate is a good metric to see if the users are interested in it. Now, there's no "good" universal percentage to decide what push-up rate is good so it's a whole different problem. Obviously, the more the better. You can track the metric weekly or monthly and compare to see if it's doing better or worse. A good thing to do is to track how many users that used the feature at least once keep on using the feature. I mentioned only a few strategies to track the success, but there are a lot more.

2.2 Which categories perform the best according to this metric?

- Total (all categories combined): 3.20555% push-up rate.
- Best category performance is `OUTDOOR_TOYS` with 7.239%. Other top ones are `TRADITIONAL_NATIONAL_CLOTHING` with 7.1599%, and `STROLLERS` with 6.8247%.

```
categories_df.sort_values('cat_push_up_rate', ascending=False)
```

	number_of_listings	revenue_from_push_ups	number_of_push_ups	cat_push_up_rate
category_2				
OUTDOOR_TOYS	221	32.0	16	7.239819
TRADITIONAL_NATIONAL_CLOTHING	838	120.0	60	7.159905
STROLLERS	52896	7220.0	3610	6.824713
MEN_FOOTWEAR	163919	21012.0	10506	6.409263
MEN_OUTERWEAR	50244	6434.0	3217	6.402755
FOOTWEAR	1056428	126420.0	63210	5.983370
CHAIRS	24648	2882.0	1441	5.846316
BAGS_BACKPACKS	394951	44380.0	22190	5.618418
BABY_FURNITURE	37920	4140.0	2070	5.458861
MEN_PULLOVERS_SWEATERS	899	94.0	47	5.228031
MEN_SHOES_NEW	35548	3630.0	1815	5.105772
COSMETICS_AND_BEAUTY_ITEMS	2215	188.0	94	4.243792

- Best subcategory performance is **MEN_FORMAL** with 18.1818%. Other top ones are **SPORT_STROLLERS** with 9.2374%, and **UMBRELLA_SHAPE_STROLLERS** with 9.0259%.

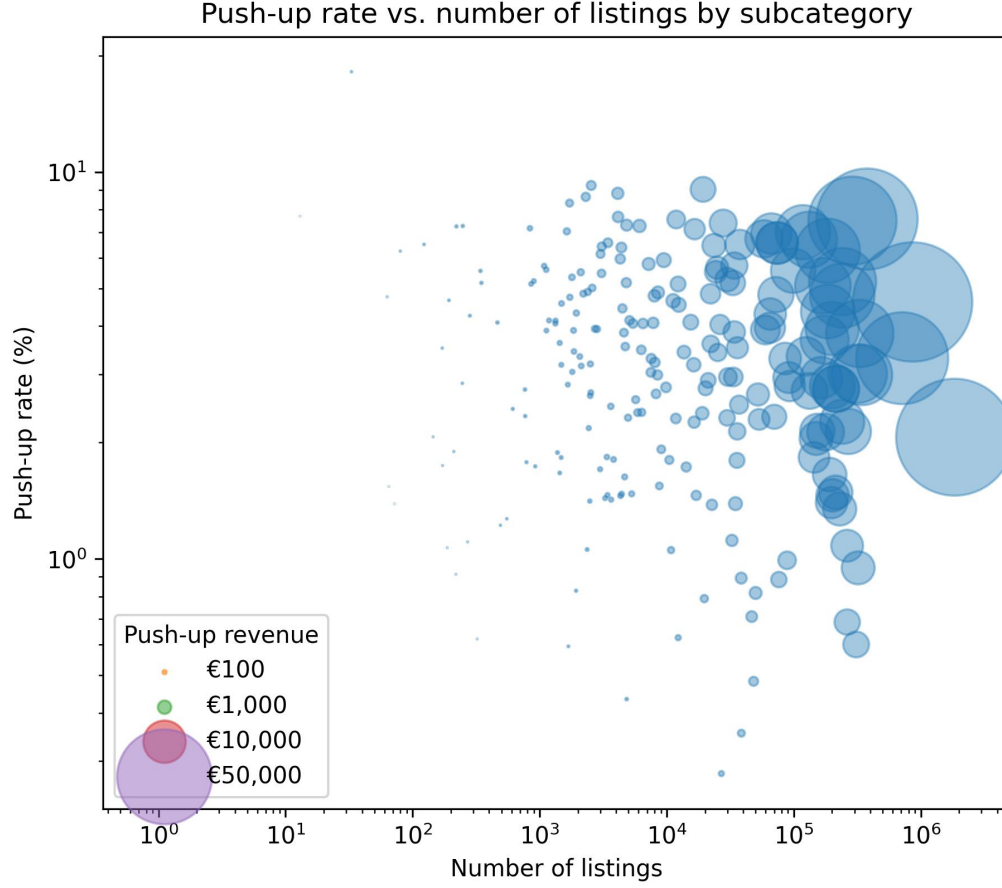


Figure 1: Push-up rate vs. number of listings by subcategory with revenue included

Some smaller subcategories have high push-up rates, but because of their low number of listings, their revenue tends to be lower. Larger subcategories tend to generate way more push-up revenue because of their large number of listings.

2.3 Correlation

An interesting correlation to check for: push-up rate vs average listing price. After calculating Spearman correlation, we get 0.769 correlation between push-up rate and average listing price. We can see that these values increase together gradually - the bigger value of subcategory's average listing price, the bigger the value of the push-up rate. This suggests that people with high-priced items are more inclined to use a 2 euro push-up feature. Since the item's price is high, it's likely that they do not mind spending 2 euros. If the item you're selling costs a low amount (e.g. 5 euro), the price of the feature may seem a lot and not exactly worth it.

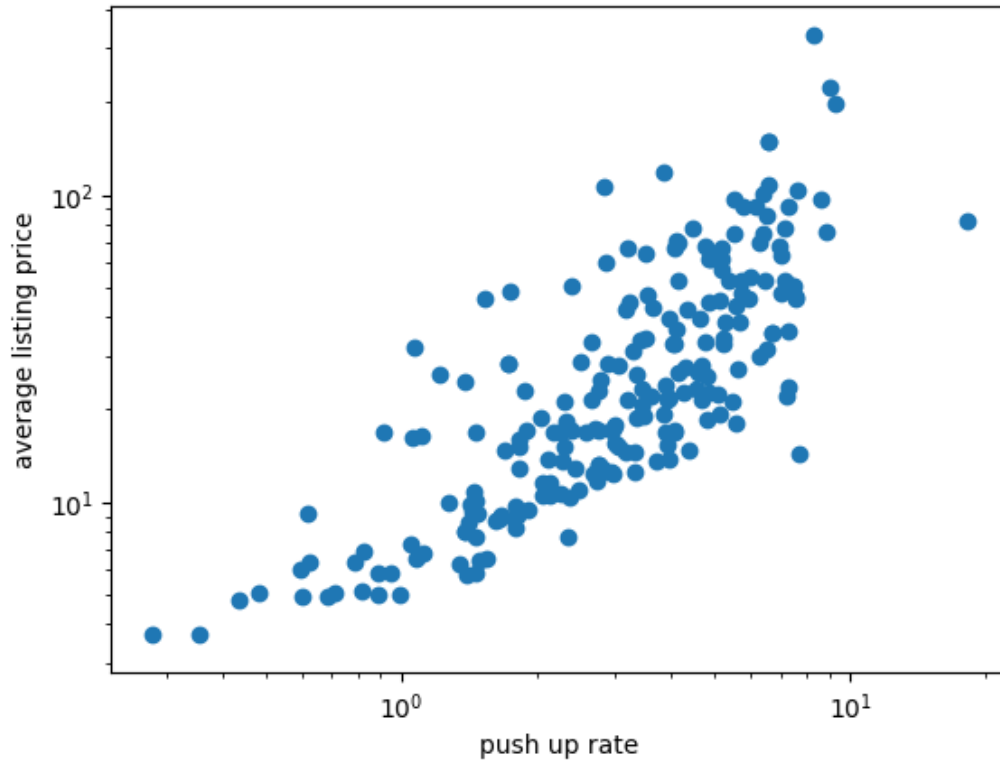


Figure 2: Subcategories average Listing price vs. Push-up rate (log scale).

3 Section III: Analyse alternative strategies

3.1 There's suggestion of setting a fixed price for the "push-up" feature different from €2. What implications will a price increase / decrease have on the business, e.g. visibility of listing on the platform?

Price increase / decrease for the "push-up" feature will have different effects on different categories and subcategories. For example, some subcategories have zero revenue from push-ups, so that means no one is using a push-up. Let's say the "push-up" feature price is decreased. That may convince them to start using the feature and that will bring revenue. On the other side, there are categories that have a lot of listings and people that use push-ups quite regularly. If the price is decreased, at first even more people will start using the feature and the front-page will become oversaturated with push-ups. Because of that, some people will start noticing that the push-up feature is no longer worth it and stop buying it. If the "push-up" feature price increased, some people will stop using it because of the price increase. Now, less people use the feature so there is less competition and the front-page is not as saturated, so you could say that buying the feature is worth it. The company may receive less one-time feature buyers, but may keep the consistent buyers. All of these theories need actual testing to see what happens and what strategy is the most optimal for bigger revenue.

3.2 Proposed A/B test for a price change

For push-up feature price change one of the tests that you could do is A/B testing. Our current problem is that we want to change the price of push-up feature to increase revenue. Let's say our hypothesis is that a lower feature price generates more revenue per eligible listing. We create two feature versions - A: original feature price of 2 euros, B: feature price lowered by 50 cents to 1.50 euro. Then we execute this testing by randomly splitting marketplace users in half and showing one group version A and another group the B version of the feature. After running the experiment

for long enough, collect and analyze the results. Let's say version A generated €110, 1000 eligible listings so that's

$$110/1000 = 0.11$$

euro per eligible listing. Version B generated €80, 640 eligible listings, so that's

$$80/640 = 0.125$$

euro per eligible listing. Version B generated more revenue per eligible listing. Before choosing version B for all users, the experiment should also have a large sample size and run for enough time to avoid noise and make the comparison reliable. If version B does not outperform version A, we can test different prices or alternative strategies.

4 Section IV: Conclusion

Overall push-up rate is approx. 3.21%, largest sub-categories push-up rate is approx. 18%, 9.2%, 9.0% and lower. Higher average listing price tends to mean more push-up usage and large sub-categories matter more for revenue. Any pricing change should be validated with testing and experimentation.